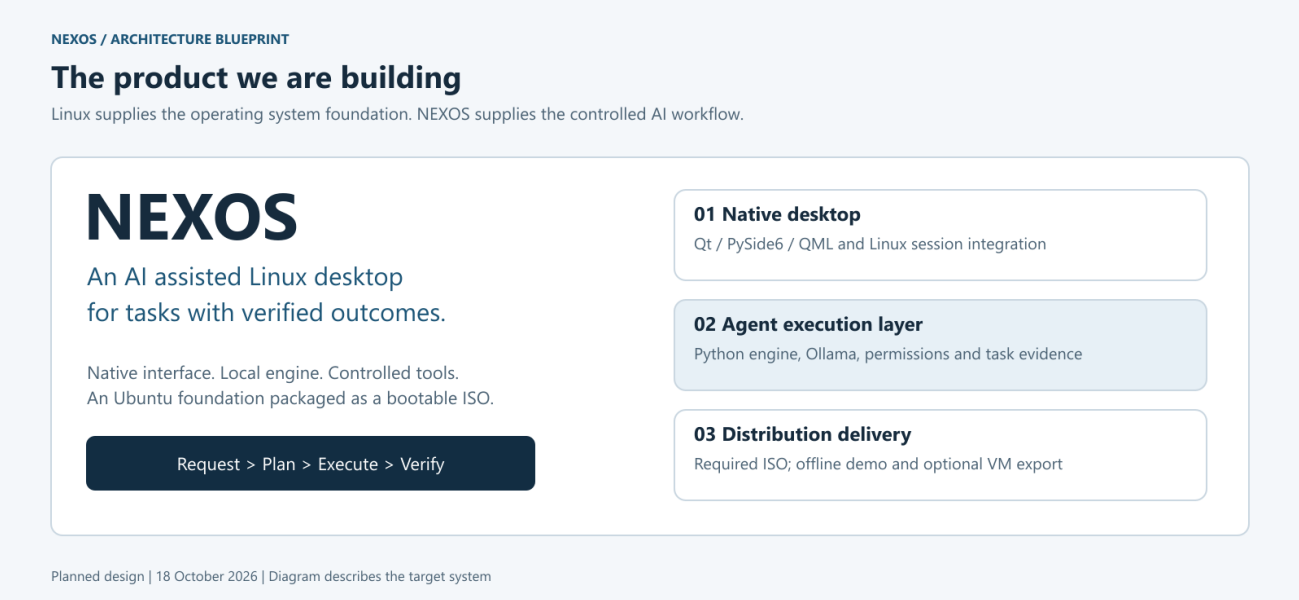
NEXOS Presentation Script

Project proposal presentation for 20 October 2026

Use the main script for approximately 10 to 12 minutes, allowing pauses. The timing is a rehearsal estimate. Speak the paragraphs, use the headings as cues and keep the reference links for questions. No live project demonstration is required.

The presentation describes the proposed complete system. The repository now begins with a small deterministic command line foundation; that is an implementation checkpoint, not the finished AI desktop or ISO.



Main presentation

1 Introduction and project title

Good morning, respected faculty members and everyone present. Today I am presenting our major project, NEXOS, an AI assisted Linux distribution for verified workstation automation.

The idea is to let a user describe an outcome in natural language and have the system carry out the required steps within clear permissions. For example, instead of manually locating a project, checking its environment and preparing a report, the user could ask NEXOS to perform that workflow and show the evidence of what was checked.

Our final deliverable will be a bootable ISO containing the Linux desktop, the NEXOS native assistant, its background execution engine and a local inference runtime.

2 Problem statement and objective

Many computer tasks still require repeated interaction across folders, terminals, browsers and applications. Even when a user knows the goal, they must understand the steps and switch between tools to complete it.

AI assistants can help with these activities, and computer agents already exist. The difficult engineering problem is making execution reliable, permission controlled and observable on a real machine.

Our objective is to build a bounded Linux workstation environment where a natural language request becomes a validated plan, authorized tool actions and independently verified results. We want to measure whether this reduces manual effort and completion time without allowing actions outside the user's permission.

3 What we mean by an AI operating system

We are building on Ubuntu, using the existing Linux kernel, drivers, networking, package management and GNOME desktop. We are developing the AI execution layer and integrating it into a customized distribution.

The contribution is in the native assistant, planning, specialist agent roles, application scheduler, permission broker, memory, recovery and verification. Linux continues to manage hardware and operating system processes.

This scope makes the project feasible for a final year team while still involving system integration and measurable engineering work. Packaging the integrated environment into an ISO will make it possible to boot and install the same system in a virtual machine.

4 An example that explains the idea

Consider the request: find my demo API project, inspect its runtime and dependency manifest, and save a diagnosis report.

NEXOS first identifies the project inside folders the user has granted access to. It then proposes a sequence of checks, runs the registered diagnostic tools and creates the report in an approved location. A

separate verifier checks that the report contains the required findings before the task is marked successful.

A second example is scheduling a project review with a Google Meet link. NEXOS will resolve the date, timezone, duration, calendar and attendees. It will show the event details, obtain authorization for sending invitations, call the Google Calendar API and retrieve the result to verify it.

These examples illustrate the same principle: a task is complete when the requested state has been checked, not simply when the model says it has finished.

5 Important features

The first feature is an intent based native interface. The user states the outcome instead of remembering a sequence of commands.

The second is specialist agent roles. A file role handles discovery, a diagnostics role handles environment inspection, a workflow role follows reviewed procedures and a productivity role prepares connected service actions. These roles can share one model, which keeps the hardware requirement manageable.

The third is permission controlled execution. The model proposes actions, but a deterministic broker checks the tool, arguments, resource scope and current authorization.

The fourth is persistent task state and memory. We will retain useful context with its source and keep enough task state to support controlled recovery after interruptions.

The fifth is independent verification and an audit trail. Users can inspect the actions, approvals and evidence behind a result.

Finally, the distribution will support local inference through Ollama, with optional cloud inference selected according to hardware capacity and the user's data handling choice.

6 Literature review

Our design is informed by five relevant research directions.

AIOS explores shared services for language model agents, including scheduling, context, memory and access control. It informs our separation between agent roles and common execution services. [L1]

MemGPT investigates hierarchical memory management for language models with limited context. It informs how we separate current task context from longer term records and retrieved knowledge. [L2]

OS Copilot presents a framework for agents that interact with operating system tools and applications. It establishes that general computer agents are an existing research direction, and informs our use of explicit adapters. [L3]

ReAct studies the interaction between reasoning, actions and observations. It informs the task loop in which plans are updated using actual tool results. [L4]

OSWorld evaluates agents using tasks in real computer environments and execution based checks. It informs our decision to judge task success using the resulting system state. [L5]

Our project focus is to integrate selected ideas into a bounded Ubuntu workstation, with enforceable tool permissions, a native desktop, recovery and reproducible ISO deployment. We will evaluate our implementation rather than claim that agentic operating systems are a new field or that we have already outperformed these systems.

7 Overall architecture

The architecture has five main parts.

First is the native desktop interface, where the user submits a request and sees the plan, progress, permissions and result.

Second is the background Python engine. It interprets the request, validates a dependency graph and schedules specialist roles.

Third is the model gateway. It connects to Ollama locally, or to an optional configured cloud provider. Ollama runs a selected model; our application implements the agents and their control logic.

Fourth is the permission broker and execution layer. Only registered adapters can access files, inspect services or call external APIs, and each action is checked before execution.

Fifth is the persistence and verification layer. SQLite stores tasks and memory metadata, while independent verifiers check outcomes and record the evidence.

The complete workflow is: request, understand, plan, validate, authorize, execute, verify and report.

8 Technology stack and reasons

We have selected Ubuntu 24.04 LTS as the base so that we can work against a consistent Linux environment.

The desktop application will use Qt 6 with PySide6 and QML. Python 3.12 with asynchronous execution will power the background engine. A private Unix domain socket will connect the desktop and command line interface to that engine.

Ollama will provide local model inference. SQLite will hold persistent state and full text search, with local embeddings considered after basic retrieval works.

For system integration, we will use systemd user services, selected D Bus interfaces and filesystem watchers. Bubblewrap and tested resource limits will isolate appropriate tool processes.

We will package the application components as Debian packages and integrate those packages into a customized Ubuntu ISO using Cubic. The application will run as part of the Linux desktop environment. It does not require a hosted NEXOS web backend.

9 Safety reliability and evaluation

Files, logs and web pages can contain misleading instructions. We will treat that content as data rather than permission to change the user's goal.

Read access will be limited to granted resources. Sensitive actions, such as sending invitations or changing project files, will require an exact action grant. Root commands and broad destructive operations are outside the first release.

We will also limit retries, model calls and execution time. If an external write times out, the system must check whether it happened before retrying. A local audit hash chain provides evidence under stated trust assumptions; we will not call it tamper proof.

Evaluation will include functional tasks, permission checks, malicious content and restart scenarios. We will measure verified success, manual interactions, completion time and resource usage. Any accuracy or productivity figures will be presented as measured results once testing is complete.

10 Implementation and ISO workflow

We will start with a deterministic diagnosis workflow, typed tools, scoped access, a report writer and a verifier. This establishes that the execution path works before introducing model driven planning.

Next, we will connect Ollama, add validated plans and agent scheduling, and build the native Qt interface and user service integration.

In parallel with those foundations, we will prepare a small boot prototype to confirm the image workflow. The full release ISO will be assembled after the application packages work on a clean Ubuntu installation.

The image pipeline will verify the Ubuntu base, install the NEXOS packages, configure the desktop and services, assemble the ISO, and test both live boot and installation. After installation, we will remove the ISO and reboot from the VM disk to check persistence.

The Core image will set up a selected model after installation. An offline demo variant can include one tested model and sample projects. The inference runtime, engine and tools will run inside the NEXOS guest.

11 Future scope and closing

After the core is reliable, useful additions include voice and multilingual input, semantic file retrieval, scheduled workflows, action previews with reversible changes, and reviewed connector plugins.

One especially useful direction is reusable routines. A user could review a workflow once and authorize a tightly scoped recurring action, while still requiring fresh approval for effects outside that scope.

Our first release will prioritize a small number of complete, testable workflows. The expected result is a bootable Linux environment that helps users complete tasks with clear permission boundaries and evidence of the result.

That is the NEXOS proposal. Thank you. I am ready to discuss the architecture, feasibility and implementation plan.

Three minute fallback script

Good morning. Our major project is NEXOS, an AI assisted Linux distribution for verified workstation automation.

The problem we are addressing is the repeated manual work involved in moving between files, terminals and applications. Our goal is to let a user describe an outcome and have the system carry out a bounded workflow with permissions and verification.

We will use Ubuntu and its existing Linux kernel and desktop. Our contribution is the integrated AI execution layer, consisting of a native assistant, a planner, specialist agent roles, a scheduler, a permission broker, memory and outcome verification.

For example, a user could ask NEXOS to find a project, inspect its dependency manifest and prepare a diagnosis report. The system will only access granted folders, execute registered tools and check the report before announcing success. A later connector will create calendar events with Google Meet links after the user approves the event details.

The main workflow is request, plan, validate, authorize, execute, verify and report. The model proposes actions; application code determines whether those actions are permitted.

Our literature review includes AIOS for common agent services, MemGPT for memory hierarchy, OS Copilot for computer tool interaction, ReAct for action and observation loops, and OSWorld for execution based evaluation. We will apply selected ideas to a bounded Ubuntu workstation and evaluate the implementation.

The technology stack is Qt and PySide6 for the native interface, Python for the engine, Ollama for local inference, SQLite for state and retrieval, and Linux facilities such as systemd and Bubblewrap for integration and isolation. Components will be packaged as Debian packages and integrated into a bootable ISO using Cubic.

We will build the deterministic execution path first, then connect the model, add the desktop interface and assemble the release image. The ISO will be tested for live boot, installation and persistence after reboot.

We will measure verified task success, manual effort, latency, resource usage and permission enforcement. Voice input, scheduled routines and additional connectors are future extensions after the core works reliably.

The expected outcome is a Linux desktop environment where users can delegate selected tasks and inspect the evidence of what actually happened. Thank you.

Literature review notes

Use these as discussion notes, not as additional paragraphs to read during the main talk. The project connections below are our engineering interpretation of the papers, not claims that the papers implement NEXOS.

Paper	Main idea relevant to the project	How NEXOS will use the idea
AIOS by Mei and colleagues, first preprint 2024, COLM 2025	Shared runtime services for agents, including scheduling and resource management	Separate specialist roles from scheduling, state and access checks; our scheduler remains in user space
MemGPT by Packer and colleagues, 2023	Manage limited model context through a memory hierarchy	Separate working context from sourced task history, knowledge and reviewed procedures
OS Copilot by Wu and colleagues, 2024	A framework for agents that use OS tools and applications	Build explicit adapters for selected Linux and connected workflows
ReAct by Yao and colleagues, 2022 preprint and ICLR 2023	Combine reasoning, actions and observations	Replan from bounded tool observations while keeping authorization outside the model
OSWorld by Xie and colleagues, 2024	Real computer tasks with execution based evaluation	Create a smaller project specific benchmark with deterministic postconditions

Our literature position is an integration and evaluation focus. Do not say that these papers lack all security, memory or scheduling, and do not use their reported performance as a NEXOS result. A project benchmark is not automatically comparable with OSWorld.

Primary references

- **L1** AIOS LLM Agent Operating System
- **L2** MemGPT Towards LLMs as Operating Systems
- **L3** OS Copilot Towards Generalist Computer Agents with Self Improvement
- **L4** ReAct Synergizing Reasoning and Acting in Language Models
- **L5** OSWorld Benchmarking Multimodal Agents for Open Ended Tasks in Real Computer Environments

References checked 19 October 2026. AIOS has a 2024 preprint and a later COLM 2025 publication; ReAct has a 2022 preprint and an ICLR 2023 camera ready version.

Likely faculty questions and short answers

Are you writing a new kernel

We are reusing Ubuntu's Linux kernel and desktop. We are developing the controlled AI execution layer and integrating it into a customized bootable distribution.

What is your contribution if Ubuntu already exists

The contribution is the native intent interface, validated workflow engine, agent scheduling, scoped permissions, persistence, recovery, verification and the measured integration into a reproducible image.

How is it different from an ordinary chatbot

NEXOS will perform registered actions and check the resulting state. Other action capable assistants already exist; our comparison will focus on the declared workflows, permissions and measured outcomes.

Why are multiple agents necessary

Specialist roles separate tools and context. They can share one model and do not require five models running in parallel. We will retain the roles only where they improve structure or measured results.

What does Ollama do

Ollama runs the selected model. NEXOS supplies the planner, tools, permission broker, scheduler, records and verification around inference.

What happens when the model makes a mistake

Invalid plans are rejected, tool requests are checked against capabilities, effects are verified and the result can be failed or partial. We will report uncertainty instead of fabricating success.

Can a malicious README control the machine

It should not be able to grant authority. Project content is untrusted data. A separate broker, scoped tools and tested isolation enforce the boundary, and malicious content is included in the evaluation.

Does it require internet

Local tasks can work offline once the runtime and model are installed. Cloud inference, calendar operations and first time model downloads require connectivity.

Will the RTX GPU work inside VMware

We will test CPU inference first and treat GPU compute exposure as unverified in the VM. A native Linux GPU experiment can be evaluated separately with supported drivers and the actual VRAM.

Is an ISO the same as a prebuilt virtual machine

An ISO is bootable media used for a live session or installation. A prebuilt VM includes an already installed system on a virtual disk. We will qualify the ISO first and can export an installed VM afterward.

What happens to data after reboot

An installed VM stores records on its virtual disk. A normal live session has temporary state unless persistence has been configured. Both paths need explicit tests and honest UI messages.

What accuracy have you achieved

The full agentic system has not been benchmarked yet. We have proposed evaluation methods and targets. We will present actual results with case counts and test conditions after implementation.

How will you prove the project works

Use defined initial states, expected effects, forbidden effects and independent verification scripts. Test functional tasks, authorization failures, malicious content, interruptions and the clean ISO installation path.

Why will you not start by building the final ISO

We can test boot integration early. The final ISO must contain working packages, so engine validation and packaging precede release image qualification. An empty branded boot image is not the completed project.

What is the present implementation status

The architecture is documented and implementation is beginning with a deterministic command line diagnosis pipeline and testable permission rules. The native desktop, model driven agents, Linux sandbox integration and release ISO remain subsequent milestones.

Features to add after the core

Best additions for the next development stage

Action preview and reversible file changes: show the target and proposed diff before a write, and keep a task specific backup for supported operations. The verifier should also check reversal. Remote invitations and other irreversible effects cannot be described as fully undoable.

Saved routines and scheduled execution: let the user review a versioned recipe and grant a limited recurring scope. Use persistent scheduling, missed run behavior and fresh approval when recipients or targets change.

A searchable evidence history: let the user inspect what was requested, which tools ran, what changed and how success was verified. This improves debugging and the final evaluation story.

Follow after the above is stable

Add semantic search with source links, push to talk voice input, regional language support, accessibility improvements and reviewed connector plugins. Each new adapter must include permissions, expected effects, failure behavior and a verifier.

Keep outside the initial release

Universal desktop control, unrestricted root automation, agent self modification and model training would expand risk and workload substantially. The initial contribution can be demonstrated with bounded workflows and a tested ISO.

One page rehearsal checklist

- **What:** an Ubuntu based distribution with an integrated native AI execution layer.
- **Why:** reduce repeated manual work while preserving control and evidence.
- **How:** request to plan to authorization to tools to verification.
- **Main stack:** Ubuntu, Qt and PySide6, Python, Ollama, SQLite, systemd and Bubblewrap.
- **Deliverable:** native packages integrated into a bootable ISO; offline demo and VM export follow their own tests.
- **Research:** AIOS for common services, MemGPT for memory, OS Copilot for adapters, ReAct for action and observation, OSWorld for evaluation.
- **First workflow:** project manifest inspection and a verified diagnosis report.
- **Measure:** verified success, manual interactions, latency, resource use and forbidden effects.
- **Current stage:** first deterministic foundation, with full agentic desktop and image integration still to build.
- **Avoid:** claiming a new kernel, universal automation, zero risk or achieved accuracy without results.

Rehearse the opening and closing aloud. If time is short, use the three minute script and keep the literature and Q and A pages available for discussion. You can explain the architecture verbally without opening the repository or showing a live project.